

Videnti Clinical AI: Agentic Architecture

1. System Overview

Videnti uses an **Agent-Facilitated RPA (AF-RPA)** model. It separates high-level planning from low-level execution using the Model Context Protocol (MCP).

2. Technology Stack

- **Orchestrator:** Antigravity (Mission Control for Agents).
- **Reasoning Engine (Cloud):** Gemini 2.5 Flash / Claude 3.5 (for Planning & Instruction generation).
- **Security Engine (Local):** Ollama running specialized medical or reasoning models via Local MCP (for PHI summarization). Preferred models include:
 - DeepSeek-R1 (7B or 14B): Best-in-class reasoning for complex clinical logic and risk flagging.
 - MedGemma-27B or MedGemma-9B: Google's clinical-specific weights for high-accuracy medical Q&A.
 - Llama-3.3-70B (Quantized): High-fidelity instruction following for structured PHI extraction.
 - Qwen2.5-7B/14B: Superior at structured JSON outputs and multi-step clinical summaries.
- **Execution Engine (Local):** Python-based MCP Servers using FastMCP and PyAutoGUI.
- **Research Engine:** ResearchAgent using Tavily and GitHub API MCP servers.

3. Data Flow

1. **Planning:** Main Agent checks the eCW schedule via the ecw-rpa-mcp.
2. **Context Gathering:** Main Agent requests a "Patient Snapshot" via MCP.
3. **Local Analysis:** Snapshot is sent to **Ollama MCP**. Local LLM flags risks (e.g., BMI > 30, history of snoring).
4. **Validation:** Research Agent checks local guidelines.md (updated via internet research) to ensure CPT criteria are met.
5. **Human Approval:** Recommendation appears in the Videnti Dashboard.
6. **Execution:** Upon approval, the Main Agent instructs the ecw-rpa-mcp to write back to eCW.

4. MCP Server Definitions

- **Ollama-MCP:** Exposes generate_summary and check_clinical_rules.
- **eCW-RPA-MCP:** Exposes scrape_schedule, open_chart, extract_text, and create_task.
- **Search-MCP:** Exposes search_github_repos and fetch_medical_guidelines.